

Production readiness — architecture questions

Platform-level questions for the CTO: tech stack, layers, database and schema, tenancy, security, the AI layer, the decision layer and operations. Each is grounded in what the code does today.

How to use this

- **Section A** lists production blockers found in the code. These need fixing whatever the answers — use it to agree priority, not to debate.
- **Section B** has the 51 questions in 13 areas. Each shows why it matters and what the platform does today, so the discussion starts from facts.
- **Short on time?** Ask the ten “decide first” questions below.

Decide first (10 questions)

#	Question	Why it can't wait
1.2	Windows VMs with .bat scripts, Linux, or containers (Docker / Kubernetes) for production?	Decides how processes are supervised, restarted, scaled and patched. Containers also make federated per-customer installs repeatable.
3.6	Should tests run on PostgreSQL only (drop SQLite), given we rely on PostgreSQL features?	Tenant isolation (RLS) and future date constraints only exist in PostgreSQL; SQLite tests can pass while production breaks.
4.1	Federated installs: do we still run row-level security inside each install as defence in depth, and complete the remaining items (tenant_id NOT NULL, RLS on users)?	Keeps isolation guaranteed if installs are ever consolidated, and closes the known gaps.
5.1	Identity: institutional SSO (OIDC / SAML via Entra ID, Shibboleth), MFA for staff and admins, and what remains of local passwords?	Universities expect SSO; MFA is a standard control for admin roles.
5.2	Token design: keep HS256 with one shared secret, or move to asymmetric keys (RS256 / ES256) with rotation? Separate keys for tokens and webhooks?	One shared secret signs both logins and webhooks; rotation is hard and a leak is total.
6.1	Where may AI models run: third-party APIs, OCI Generative AI inside the tenancy, or self-hosted? Per customer choice?	Prompts contain student data. This is a data-protection decision, not a technical preference.
6.3	Do we formally adopt the rule “AI suggests, rules and people decide” — AI never writes a record without a human or deterministic step?	Gives a clear, defensible statement for the client and regulators; most features already follow it.
7.1	Which rules stay in code and which become configuration per institution (regulations, progression, grading, lifecycle arithmetic, statutory validation)?	Every customer has different regulations. Rules in code mean a release per customer change; rules in configuration need governance and versioning.
8.1	Background jobs: keep an in-process scheduler with job locking, or adopt a queue (PostgreSQL-based or Redis / Celery / ARQ)?	Today exactly one worker may run; there's no failover and a second copy duplicates email.
12.1	CI/CD pipeline: which platform, and which gates (lint, types, tests on PostgreSQL, security scan, migration check) must pass before merge and deploy?	The strongest evidence for code quality and the basis for safe releases to many installs.

A. Production blockers found in the code

Found in a code review on 30 September 2026. None needs a strategic decision to fix; they need scheduling before any production go-live.

#	Finding	Where	Risk	Fix
1	Token-signing secret defaults to <code>change-me-in-prod</code> ; nothing stops the app starting with it. The same secret also signs inbound webhooks	<code>core/config.py:20</code>	Anyone who knows the default can forge logins; one leaked key breaks both auth and webhooks	Refuse to start in production with the default; separate keys per purpose; move to a secrets store
2	Backend started with <code>--reload</code> (development auto-restart), single process	<code>start-backend.bash:39</code>	Restarts on file changes, slower, not built for production load	Production process manager, several workers, no reload
3	No CI pipeline; no linting / type checks on the backend; tests run on SQLite, and the row-level-security tests are skipped without PostgreSQL	repo root, <code>tests/</code>	The tenant-isolation guarantee isn't tested on every change	CI on every push: lint, types, full test suite against PostgreSQL
4	Refresh token (14-day) kept in browser <code>localStorage</code> ; no <code>Content-Security-Policy</code>	<code>AuthContext.tsx</code>	A single script-injection bug could steal long-lived sessions	<code>httpOnly</code> secure cookie for the refresh token; add a CSP
5	Login rate limit stored in memory per process	<code>security_middleware.py</code>	Resets on restart; not shared across workers or servers	Shared store (database or Redis) or the proxy
6	Uploaded files on local disk; generated returns stored in a database text column; no virus scanning	<code>core/storage.py</code>	Lost if the VM is lost; no malware check on uploads	Object storage (OCI Object Storage) with scanning; returns stored as files
7	No backup scripts (restore runbook only); encryption at rest not configured	<code>deploy/</code>	Data loss and a failed security review	Automated, tested backups; encrypted storage
8	No error tracking, metrics or tracing (settings exist, nothing wired)	<code>core/config.py</code>	Problems in production are found by users, not by us	Error tracking, metrics, alerts
9	Exactly one background worker may run; two would double-send email; no job locking	<code>worker.py</code>	No failover for scheduled jobs; a second copy causes duplicates	Job locking (e.g. PostgreSQL SKIP LOCKED / advisory locks) so workers can scale and fail over
10	AI features send prompts to third-party providers; real names are sent unredacted (by design, noted in code)	<code>core/llm/governance.py:12</code>	Personal data leaves the platform	Decision needed (see AI layer questions)
11	API docs (<code>/docs</code> , OpenAPI) always on	<code>main.py:24</code>	Publishes the full API surface to anyone	Off in production, or behind login
12	No MFA; single sign-on settings exist but unused	<code>identity/</code>	Weak for admin accounts; universities expect SSO	MFA for privileged roles; SSO via OIDC / SAML

What is already solid

Area	In place
Tenant isolation	PostgreSQL row-level security with FORCE on ~95 tables; fail-closed role available
Sessions	Short access tokens; refresh tokens stored server-side, revocable, rotated on use; login lockout
Audit	Every write recorded in an audit log, plus richer domain audit entries
API consistency	Versioned prefix, one error format with request IDs, consistent pagination
Reliable integration	Transactional outbox with retries and dead-letter; signed webhooks
AI safety	Every AI feature has a tested non-AI fallback; per-tenant kill switch; token budgets; human approval on AI-drafted actions
Read scaling	Separate read sessions that can point at a replica
Tests	575 automated tests

B. Questions by area

1. Tech stack and runtime

#	Question	Why it matters	Today
1.1	Is the stack (FastAPI / Python, SQLAlchemy async, PostgreSQL, Next.js 14) confirmed as the long-term platform, and which versions do we pin and support?	Every other decision builds on it; version policy decides upgrade and security-patch cadence.	Python + FastAPI, PostgreSQL 16+ (18 in use), Next.js 14; no documented version policy
1.2	Windows VMs with .bat scripts, Linux, or containers (Docker / Kubernetes) for production?	Decides how processes are supervised, restarted, scaled and patched. Containers also make federated per-customer installs repeatable.	Windows .bat scripts on the VM; a Dockerfile and docker-compose exist but aren't the deployment path
1.3	How many API processes and servers per install, behind which proxy / load balancer?	A single process is a single point of failure and limits throughput.	One uvicorn process with --reload; Caddy proxy with TLS
1.4	Do we introduce Redis (or similar) for rate limits, locks, caching and queues, or keep everything in PostgreSQL?	Several gaps (shared rate limit, job locking, queues) need a shared store. PostgreSQL can do most of it; Redis is one more thing to run in every install.	Redis configured but unused; nothing shared between processes

2. Application architecture and layers

#	Question	Why it matters	Today
2.1	Do we formalise the modular monolith: enforced module boundaries (import rules checked in CI), and a published list of modules and their owners?	Boundaries that are only convention erode. Enforced rules are the evidence against the “brittle AI code” concern and keep future extraction possible.	router / service / repository / models / schemas per module; convention only, no checks; 76 routers
2.2	Which modules are candidates to become separate services later (statutory engine, integration / publication, AI), and what is the trigger to split?	Heavy work that could slow users should be isolatable. Defining the trigger avoids splitting too early.	Everything in one process; the worker is a separate process
2.3	API standards: versioning and deprecation policy, error format, pagination, idempotency keys for writes — written down as a contract?	The external object API (client ask) makes these promises to third parties; changing them later breaks integrations.	/api/v1, consistent error envelope, limit / cursor pagination; no deprecation policy
2.4	Is the frontend API client generated from the OpenAPI schema, so frontend and backend can't drift?	Hand-written clients silently drift from the API as it grows.	Hand-written client; a generator script exists but is barely used

3. Database and schema

#	Question	Why it matters	Today
3.1	Schema conventions: naming, UUID keys, required audit columns, soft delete vs hard delete, JSON columns — which are the rules?	Consistent conventions make the schema reviewable (the client asked to see it) and predictable to extend.	UUID keys, created / updated timestamps, audit-log table; JSON in ~19 models; soft delete only in a few places
3.2	Migration policy: are all migrations backward compatible (expand → migrate → contract) so upgrades don't need downtime, and who reviews them?	With many customer installs, every release runs migrations N times; a bad one is N incidents.	87 Alembic revisions, single head; migrations run automatically at backend start
3.3	Indexing and query performance: who owns it, and do we test with realistic volumes (students, history rows, custom attributes)?	Effective dating and custom attributes both multiply rows; performance problems show up at volume, not in demos.	Per-column indexes; no documented strategy or volume tests
3.4	Connection pooling: PgBouncer in front of PostgreSQL? Pool sizes per process?	More processes × pool size can exhaust PostgreSQL connections.	pool 10 + overflow 5 per process; code assumes PgBouncer
3.5	Read replicas: do larger installs get one for reports, returns and data publication?	Keeps heavy reads off the primary — the reason direct DB access was declined.	Replica URL supported (read sessions use it if set); not deployed
3.6	Should tests run on PostgreSQL only (drop SQLite), given we rely on PostgreSQL features?	Tenant isolation (RLS) and future date constraints only exist in PostgreSQL; SQLite tests can pass while production breaks.	Tests default to SQLite; PostgreSQL-only tests skip

4. Multi-tenancy and isolation

#	Question	Why it matters	Today
4.1	Federated installs: do we still run row-level security inside each install as defence in depth, and complete the remaining items (tenant_id NOT NULL, RLS on users)?	Keeps isolation guaranteed if installs are ever consolidated, and closes the known gaps.	RLS with FORCE on ~95 tables; fail-closed app role optional; two items open
4.2	Is the fail-closed database role mandatory in production?	Without it, the app connects as the table owner and a missing tenant setting is less protected.	Opt-in via APP_DATABASE_URL
4.3	Per-customer backup and restore: a whole install, or one tenant within a shared install?	Customers will ask how their data is restored and exported when they leave.	Single-tenant restore runbook exists; no backup automation

5. Security

#	Question	Why it matters	Today
5.1	Identity: institutional SSO (OIDC / SAML via Entra ID, Shibboleth), MFA for staff and admins, and what remains of local passwords?	Universities expect SSO; MFA is a standard control for admin roles.	Own login, pbkdf2 hashing, lockout after 5 failures; no MFA; OIDC settings unused
5.2	Token design: keep HS256 with one shared secret, or move to asymmetric keys (RS256 / ES256) with rotation? Separate keys for tokens and webhooks?	One shared secret signs both logins and webhooks; rotation is hard and a leak is total.	HS256, one secret with an insecure default; 15-min access / 14-day rotating refresh
5.3	Session storage in the browser: move the refresh token to an httpOnly cookie, add CSP and CSRF protection?	Standard protection against session theft through script injection.	Refresh token in localStorage; no CSP; no CSRF (Bearer tokens)
5.4	Authorisation model: are hard-coded role lists for row scoping acceptable, or do we need configurable, data-driven scoping (e.g. by department, programme)?	Institutions organise access differently; hard-coded lists mean code changes per customer.	Permission codes in the token; row scoping by fixed role lists
5.5	Secrets management: OCI Vault (or similar) instead of .env files, with rotation?	Secrets in files on the VM are a common audit finding.	.env via pydantic-settings
5.6	Data protection: encryption at rest, field-level encryption for sensitive data (e.g. disability, nationality), retention and deletion rules?	Special-category data needs stronger protection; retention interacts with never-delete history.	No field encryption; encryption at rest not configured
5.7	Security assurance: penetration test before go-live, dependency scanning, a vulnerability process?	Usually a contract condition for universities.	Hardening backlog exists (G7); no scanning in place

6. AI layer

#	Question	Why it matters	Today
6.1	Where may AI models run: third-party APIs, OCI Generative AI inside the tenancy, or self-hosted? Per customer choice?	Prompts contain student data. This is a data-protection decision, not a technical preference.	Groq / OpenRouter; per-tenant kill switch; token budgets
6.2	What personal data may be sent to a model? Should names and identifiers be pseudonymised before sending?	Code states real names are sent unredacted; only logs are redacted.	Redaction applies to logged prompts only
6.3	Do we formally adopt the rule “AI suggests, rules and people decide” — AI never writes a record without a human or deterministic step?	Gives a clear, defensible statement for the client and regulators; most features already follow it.	Deterministic fallbacks tested; intervention plans and model promotion need human approval
6.4	AI audit: should prompts, responses, model and version be stored per call for accountability?	If an AI suggestion influenced a decision about a student, we may need to show what it said.	Traces stream to the browser only; not persisted
6.5	Model and prompt versioning, evaluation before changing a model, cost limits per customer?	Model changes can silently change behaviour; costs scale with usage.	Per-user daily token budget; no evaluation suite

7. Decision layer (business rules)

#	Question	Why it matters	Today
7.1	Which rules stay in code and which become configuration per institution (regulations, progression, grading, lifecycle arithmetic, statutory validation)?	Every customer has different regulations. Rules in code mean a release per customer change; rules in configuration need governance and versioning.	Mostly service code + constants; configurable: ~15 settings, JSON grading policy, workflow definitions
7.2	Do we need a general rules engine, or extend the existing versioned workflow definitions and policy objects?	A rules engine is powerful but costly to build and govern.	Workflow definitions stored as versioned state machines; no rules engine
7.3	Should rule and policy changes be effective-dated and versioned like data (which rule applied to this student on that date)?	The CMA and appeals mean we must show which regulations applied at the time.	Grading policy is a single current JSON per programme
7.4	Every automated decision explainable and audited (inputs, rule version, outcome)?	Appeals and regulators ask why a student was marked, flagged or progressed.	Audit log records writes; decision inputs not captured systematically

8. Integration and asynchronous processing

#	Question	Why it matters	Today
8.1	Background jobs: keep an in-process scheduler with job locking, or adopt a queue (PostgreSQL-based or Redis / Celery / ARQ)?	Today exactly one worker may run; there's no failover and a second copy duplicates email.	APScheduler worker, 3 interval jobs, outbox with retries and dead-letter; no locks
8.2	Delivery guarantees for outbound events: at-least-once with idempotent consumers?	Integrations (finance, GrantsNow) must cope with retries without double-posting.	Transactional outbox, backoff, dead-letter; idempotency only in a few places
8.3	External API security: OAuth2 client credentials, scopes, rate limits per client, audit per call?	The object API will let third parties write data; it needs the same controls as users.	No service accounts or API keys for third parties

9. Files and storage

#	Question	Why it matters	Today
9.1	Object storage (OCI Object Storage) for uploads and generated returns, with lifecycle rules?	Local disk is lost with the VM; large text in the database bloats it.	Local disk; returns as text in the database; S3 option falls back to disk
9.2	Malware scanning of uploads, and allowed file types?	Universities upload documents from students.	50 MB limit; no scanning

10. Observability and operations

#	Question	Why it matters	Today
10.1	Which observability stack: error tracking (Sentry or OCI), metrics (Prometheus / OCI Monitoring), tracing (OpenTelemetry), log shipping?	Without it, production problems are found by users.	JSON logs with request IDs; health endpoints; Sentry / OTel settings unused
10.2	What are the SLOs (availability, response time) and who is on call?	Turns “production ready” into measurable targets and defines support.	None defined
10.3	Runbooks: deploy, rollback, restore, rotate secrets, disable AI, incident response?	Operators need tested procedures, especially across many installs.	Deployment guide + single-tenant restore runbook

11. Reliability, scale and performance

#	Question	Why it matters	Today
11.1	Design targets: students per customer, concurrent users, peak events (enrolment week, return deadlines)?	Sizing, load tests and pool settings all derive from these.	An earlier load test measured read p95 of 6 ms; no current targets
11.2	Backups: frequency, retention, point-in-time recovery, and a tested restore schedule?	Untested backups are the most common cause of permanent data loss.	No backup automation
11.3	High availability: is a single VM acceptable, or do we need a standby database and more than one app server?	Decides cost per customer and the uptime we can promise.	Single VM layouts documented

12. Delivery: CI/CD, testing and environments

#	Question	Why it matters	Today
12.1	CI/CD pipeline: which platform, and which gates (lint, types, tests on PostgreSQL, security scan, migration check) must pass before merge and deploy?	The strongest evidence for code quality and the basis for safe releases to many installs.	No CI; ESLint only; no ruff / mypy; no type-check step
12.2	Environments: dev / test / staging / production on separate servers, with production data never used in lower environments?	Four databases on one server risks cross-environment mistakes.	Four databases on one server, selected by DATABASE_URL
12.3	Test strategy: target coverage, end-to-end browser tests, load tests, and security tests?	Defines what “tested” means before each release.	575 automated tests (mostly integration), 1 end-to-end flow; no browser or load tests in CI
12.4	Release management: semantic versions, release notes, feature flags default off, rollback plan?	Needed for federated installs and the “features off by default” promise.	One off-by-default flag; no release process documented

13. Frontend architecture

#	Question	Why it matters	Today
13.1	Route protection on the server (Next.js middleware) as well as in the browser?	Client-side guards only hide pages; the server should refuse to render protected ones.	Client-side route guards; the API enforces permissions
13.2	Accessibility standard (WCAG 2.2 AA) and supported browsers?	UK public-sector bodies, including universities, must meet accessibility regulations.	Not defined
13.3	Internationalisation and terminology: formal i18n library and per-customer labels?	The client raised England / Scotland terminology.	Home-grown translations; no per-customer labels

Amber-marked question IDs are in the “decide first” list.

After the call

- Record each decision in docs/DECISIONS.md (next number after D-04).
- Turn section A into a hardening backlog with owners and dates; several items already appear in docs/G7_HARDENING.md.
- Use the answers to write the architecture document the client asked for.